

Create, update and retrieve records using console apps

In this lecture we create a C# console application connecting to our Dynamics 365 environment for creating, updating, and retrieving records. This represents a general method for integrating Dynamics 365 with third-party services.

Let us first recall the following concepts:

1. In computer science, the term "application" refers to a software program or set of programs designed to perform a specific task or set of tasks for end-users, businesses, or other computer systems. Applications are a fundamental concept in computing and are created to meet various needs, ranging from basic functions to complex processes.
2. In the context of console applications in Visual Studio, the term "application" refers to a software program that runs in a console window and interacts with users through text input and output. A console application typically follows a command-line interface (CLI) paradigm, where users enter commands and provide input through the console, and the application responds with textual output.
3. An API (Application Programming Interface) is a set of rules and tools that allows one software application to interact with or use the functionality of another. It defines the methods and data formats that applications can use to request and exchange information. APIs facilitate communication and integration between different software systems, services, or components.

1. Connecting with username and password

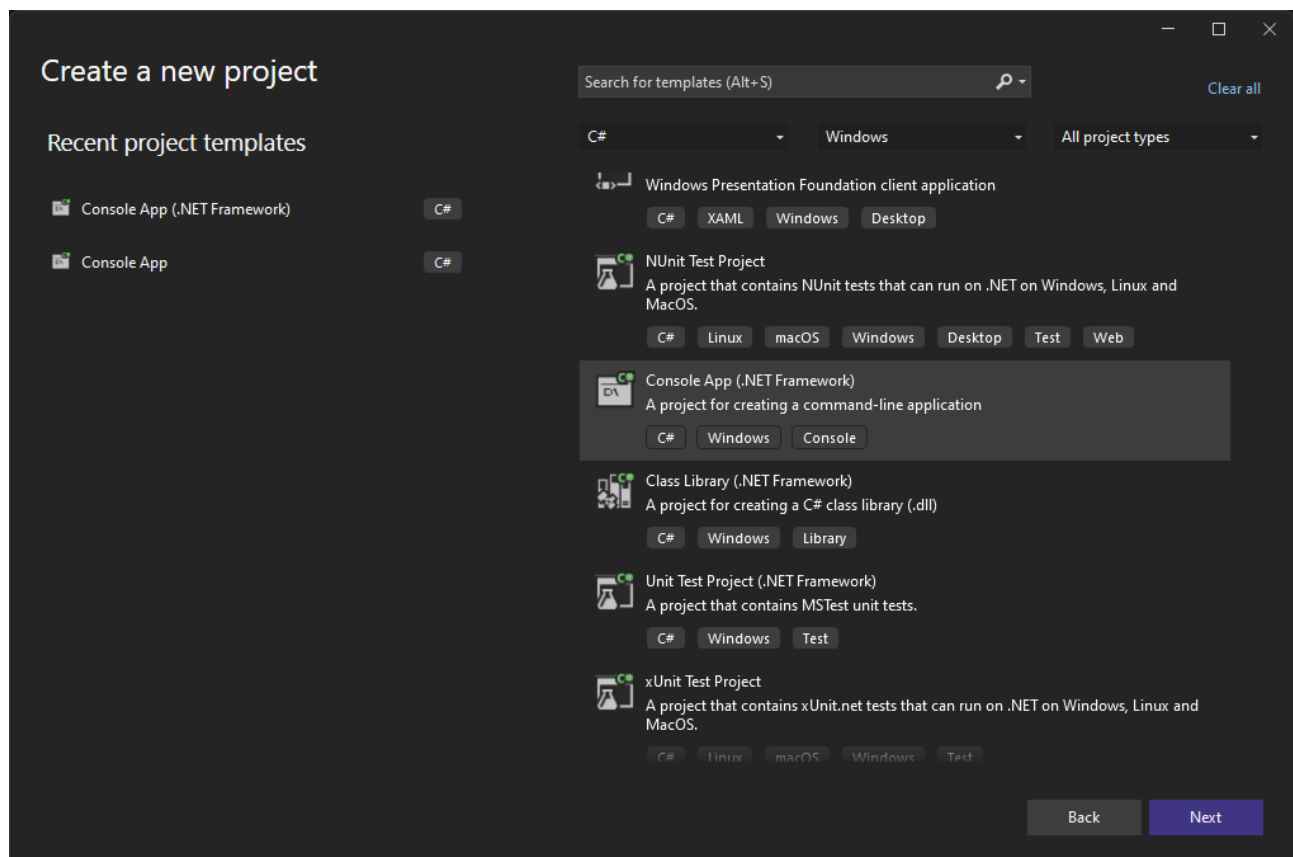
To integrate third-party services into Dynamics 365 using Visual Studio and C#, we generally follow these steps:

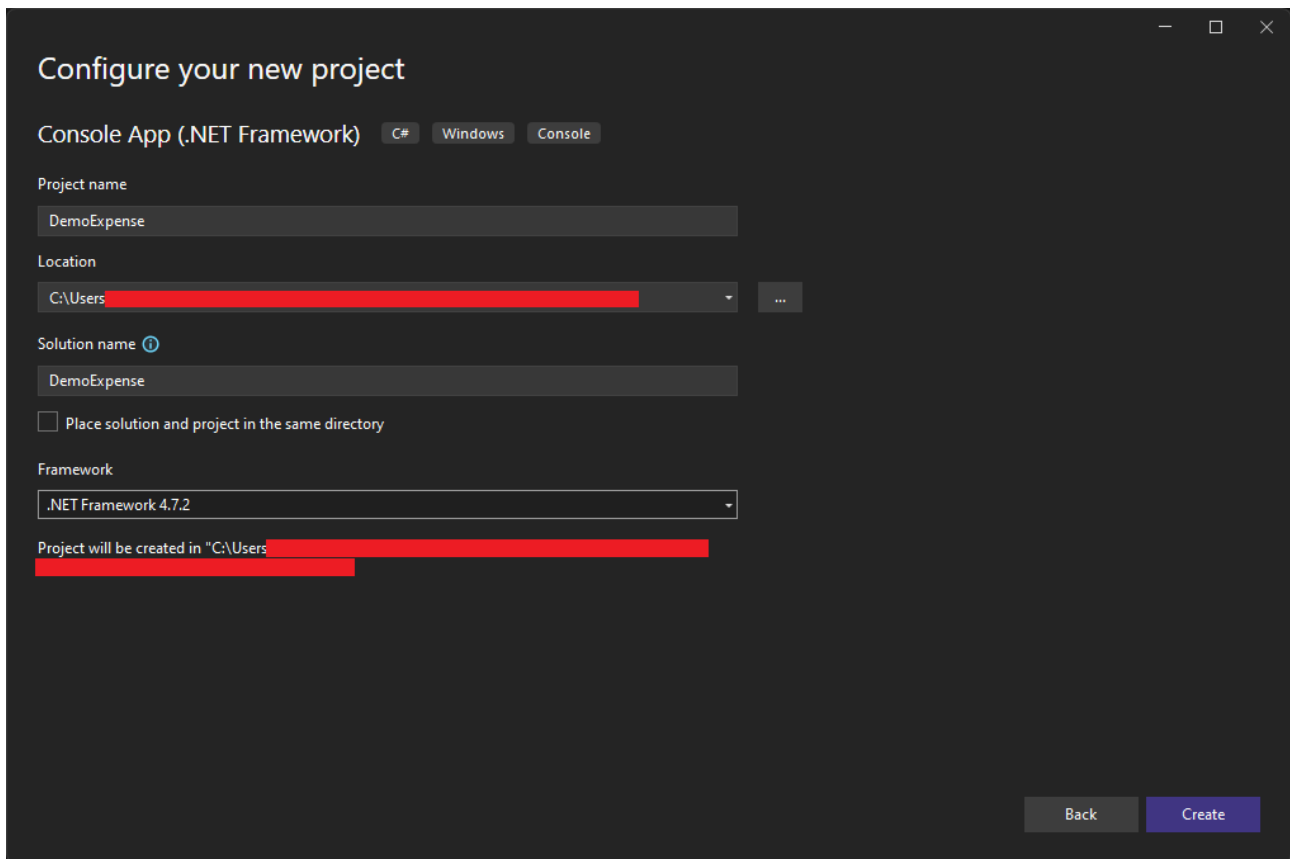
1. **Identify the Third-Party Service:** clearly define the third-party service to be integrated with Dynamics 365, understand its APIs, authentication mechanisms, and the operations it supports.
2. **Create an App Registration in Azure AD:** if the third-party service requires authentication, we may need to create an App Registration in Azure Active Directory (Azure AD). This involves obtaining a client Id and client secret or other credentials that our application will use to authenticate with the third-party service. An alternative and less secure authentication can be done by typing the username and password (of the user trying to connect) directly into a connection string.
3. **Create a Console Application (or Custom Assembly):** in Visual Studio we create a new console application or a custom assembly project based on our integration requirements.
4. **Add Necessary Dependencies:** if the third-party service provides a .NET SDK or client library, we must add the necessary dependencies to the project. Otherwise, we may need to use standard HTTP libraries (e.g., HttpClient) to make API requests.
5. **Implement Integration Logic:** write the C# code to interact with the third-party service. This includes making API requests, handling responses, and parsing data. We must ensure that we handle any required authentication and authorization steps.
6. **Handle Dynamics 365 Connection:** if our integration involves updating or retrieving data from Dynamics 365, we must ensure that we have the necessary connection logic to interact with the Dynamics 365 API. We might need to use the Dynamics 365 SDK or the Power Platform SDK, depending on our Dynamics 365 environment.

7. **Error Handling and Logging:** implement robust error handling and logging mechanisms in our code. This is important for troubleshooting and maintaining the integration over time.
8. **Testing:** test the integration thoroughly in a development or test environment. In addition, we must verify that data is transferred accurately between Dynamics 365 and the third-party service, also paying attention to edge cases and error scenarios.
9. **Deployment:** once testing is successful, we deploy our integration to the production environment. Here we must ensure that any necessary configuration changes, such as updating connection strings or API endpoints, are appropriately managed for the production environment.
10. **Monitoring and Maintenance:** implement monitoring mechanisms to track the performance and health of the integration, being prepared to address any changes in the third-party service's APIs or authentication methods that may affect the integration.
11. **Security:** pay attention to security considerations, especially when dealing with sensitive data. Use secure practices for storing credentials, encrypting data in transit, and validating inputs.

So, to create the application we first have to set up a new **console application** project in Visual Studio. In doing that we select the .NET Framework, which provides a programming model, a vast library of pre-built code, and a runtime environment for executing applications.

It's worth noting that, instead, a class library project is commonly used for developing Dynamics 365 plugins. A **plugin** represents a custom business logic that executes in response to specific events or messages within the system, such as the creation, update, or deletion of records.





Configure your new project

Console App (.NET Framework) C# Windows Console

Project name
DemoExpense

Location
C:\Users\...

Solution name ⓘ
DemoExpense

☐ Place solution and project in the same directory

Framework
.NET Framework 4.7.2

Project will be created in "C:\Users\..."

Back Create

It is advisable to opt for a .NET Framework version within the range of 4.6 to 4.8 for optimal performance and compatibility. This is because, in trying to connect to Dynamics 365, we might have to download a couple of dlls. An old version of the .NET Framework might not be updated with the most recent dlls necessary for integrating a service, for example Stripe.

Dynamic Link Libraries (DLLs) are files used in Windows operating systems to store code and data that can be shared among multiple programs. They support dynamic linking, allowing code to be linked at both compile-time and run-time. DLLs help in promoting code reusability by providing a means for different applications to utilize the same functions or procedures. The code within a DLL is not loaded into memory until it is specifically called by a program, allowing for efficient use of system resources. DLLs can be linked to a program during run-time, offering flexibility and adaptability. They are commonly used to extend the functionality of applications, with each DLL file having a ".dll" file extension. Overall, DLLs contribute to modularizing code, reducing redundancy, and supporting the shared use of code components across various software applications.

The following is the initial body of the program. Notice the using statements, which are necessary for resource management and to create namespace aliases.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

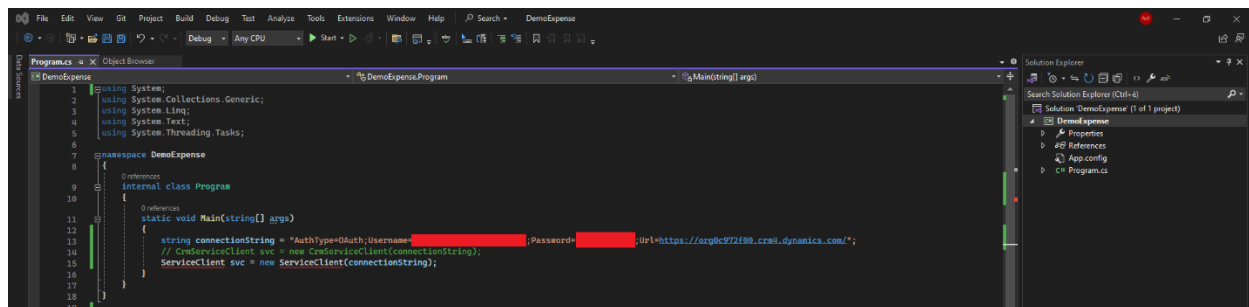
namespace DemoExpense
{
    internal class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

To access Dynamics 365 from our application we need a connection string. A **connection string** in the context of accessing Dynamics 365 from a C# console application serves as a configuration parameter that contains information required to establish a connection to the Dynamics 365 environment. It typically includes details such as the server address, authentication credentials, and other relevant connection parameters. The connection string is used by the application to establish a connection to the Dynamics 365 instance and interact with its data.

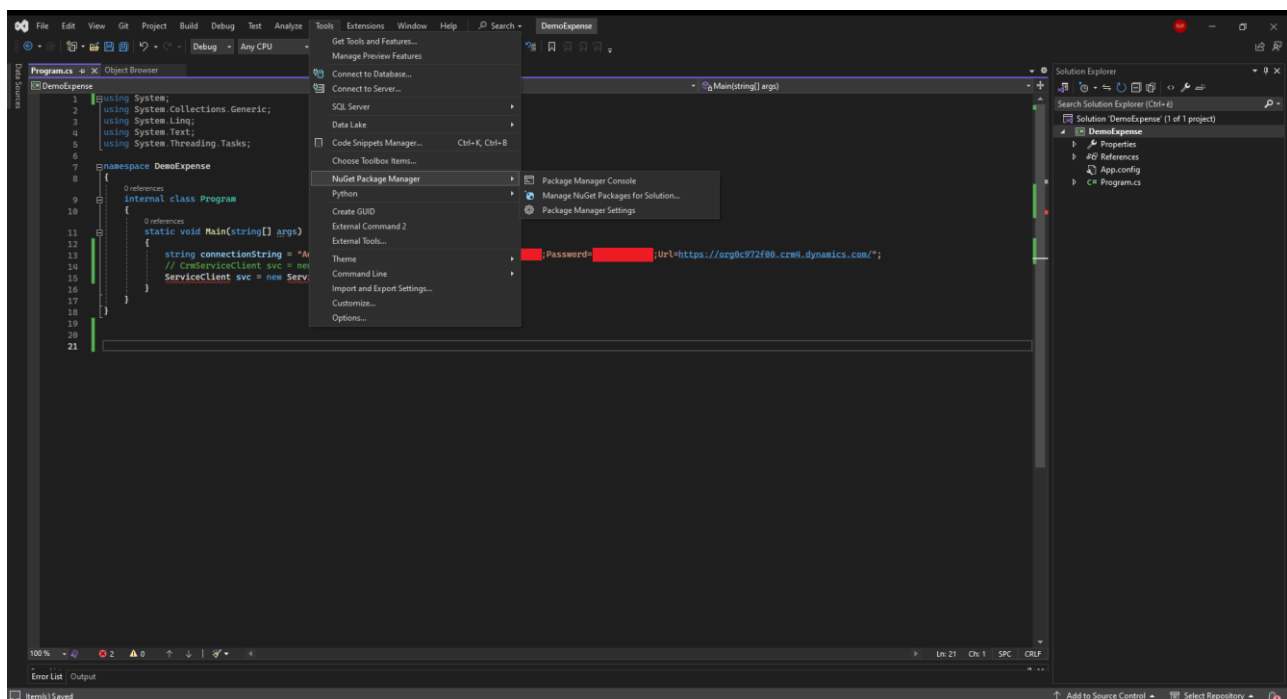
Here is an example of a connection string using username and password:

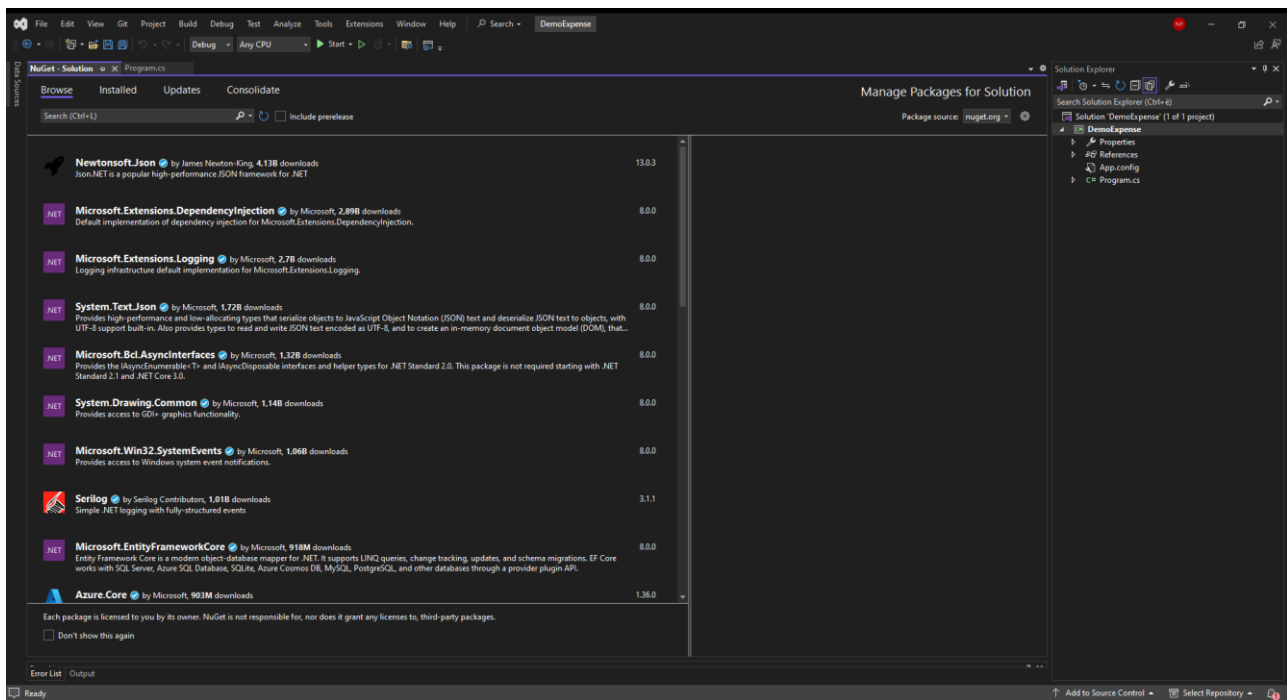
```
string connectionString =  
"AuthType=OAuth;Username=<username>;Password=<password>;Url=https://<organization>.crm.dynamics.com";
```

Navigate to <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/xrm-tooling/use-connection-strings-xrm-tooling-connect> for extra documentation about setting up a connection string like that of the following example.

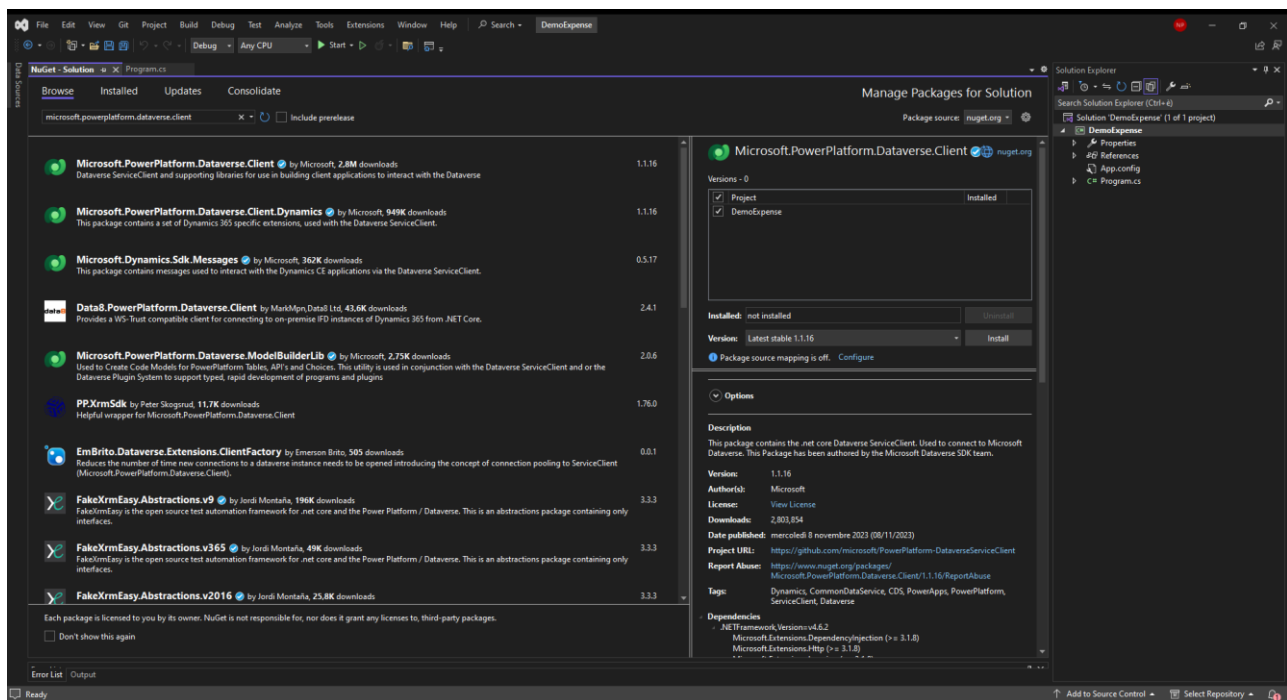


Notice that ServiceClient, necessary to establish the connection, is not a type already defined in Visual Studio. To obtain it we have to go to Tools, NuGet Package Manager and Manage NuGet Packages for Solution....

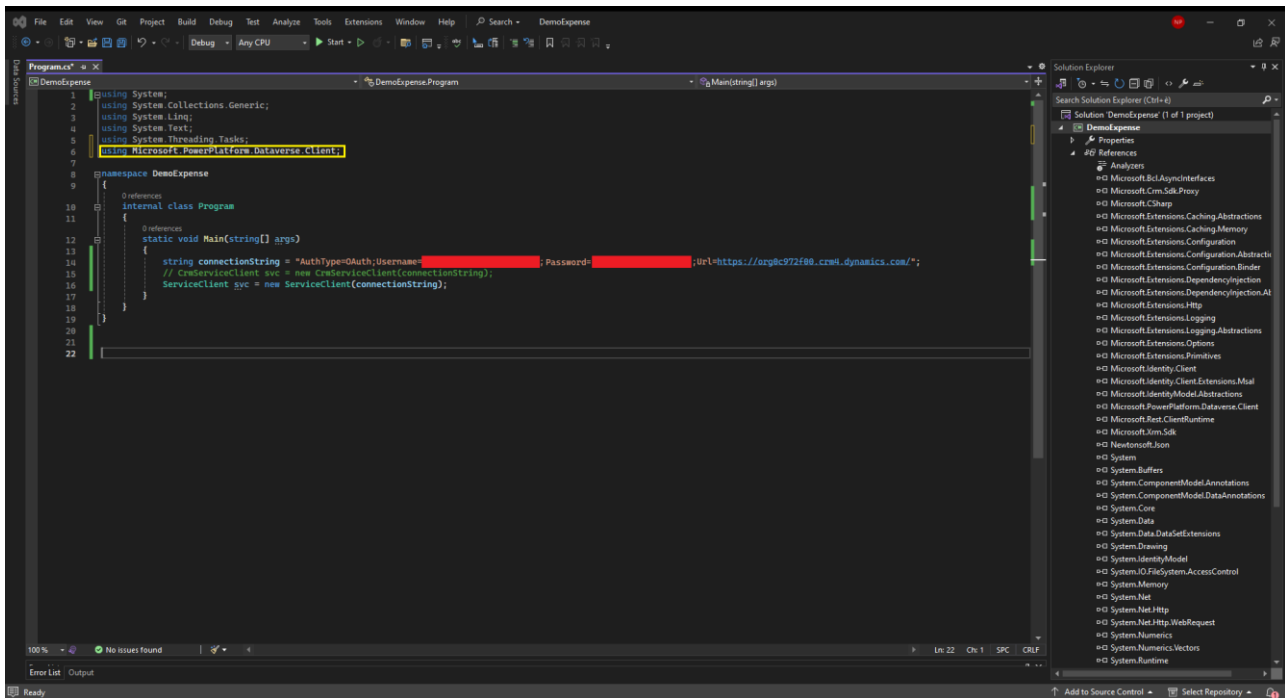




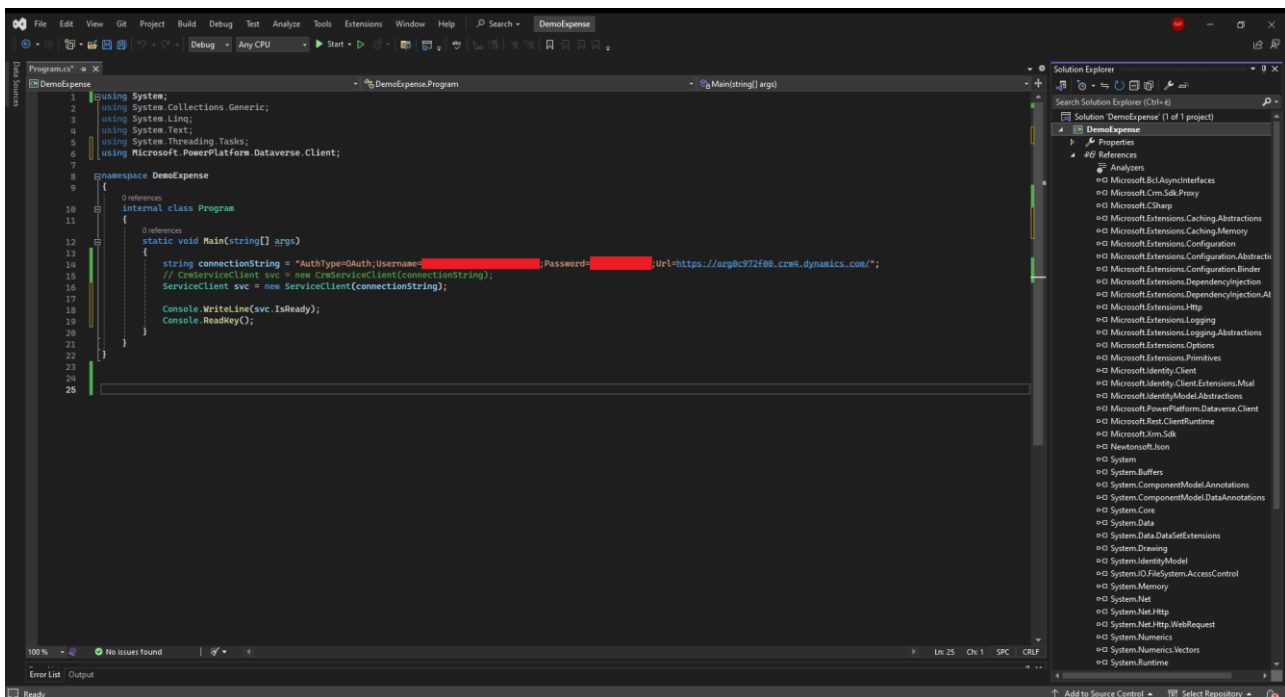
In the search bar we look for the Microsoft.PowerPlatform.Dataverse.Client package and install it.



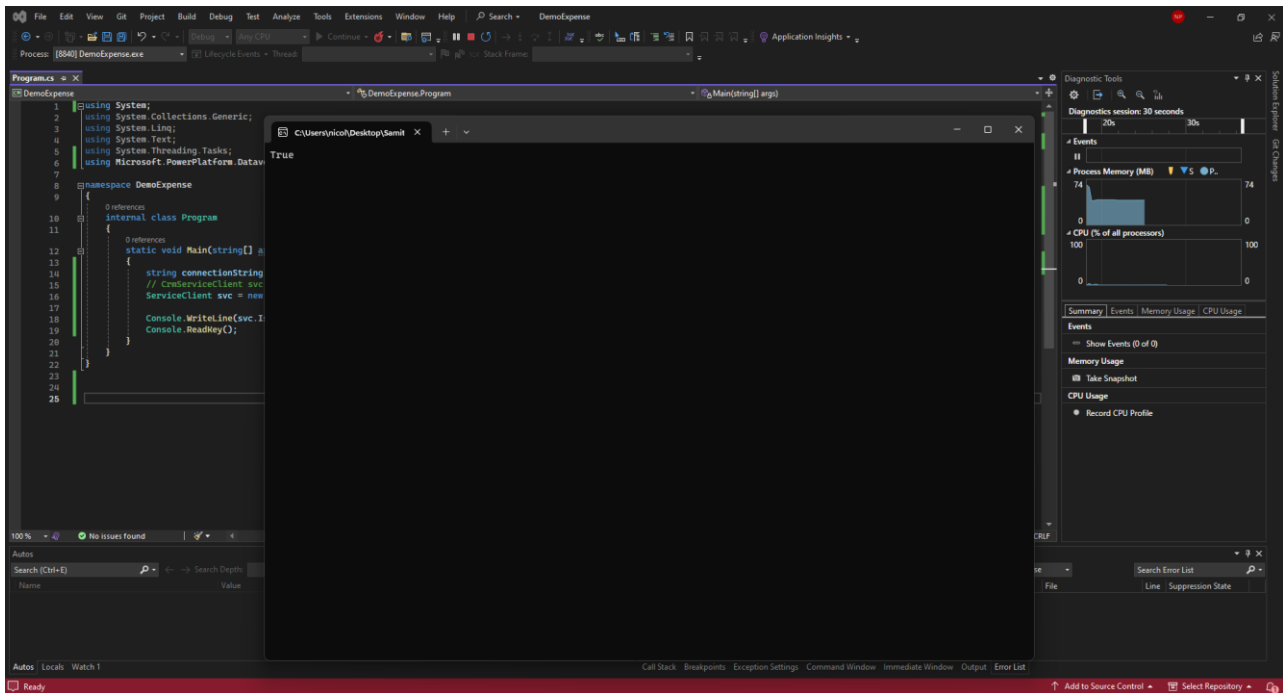
This operation downloads a bunch of objects (notice the list under References on the right). Next, we add the using statement to effectively use the package we just installed in our application.



To be sure that we are connecting correctly, we also add a Console.WriteLine printing true or false.



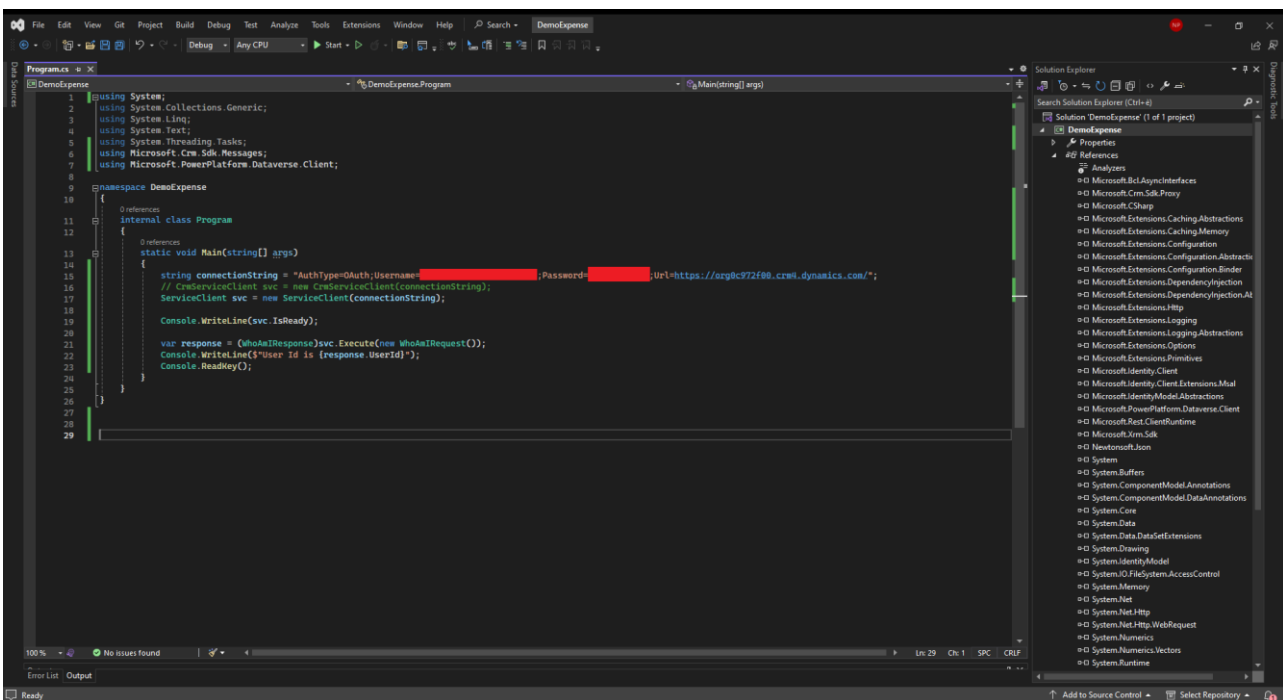
If everything is correct, when we start the application, we should see true on the console.

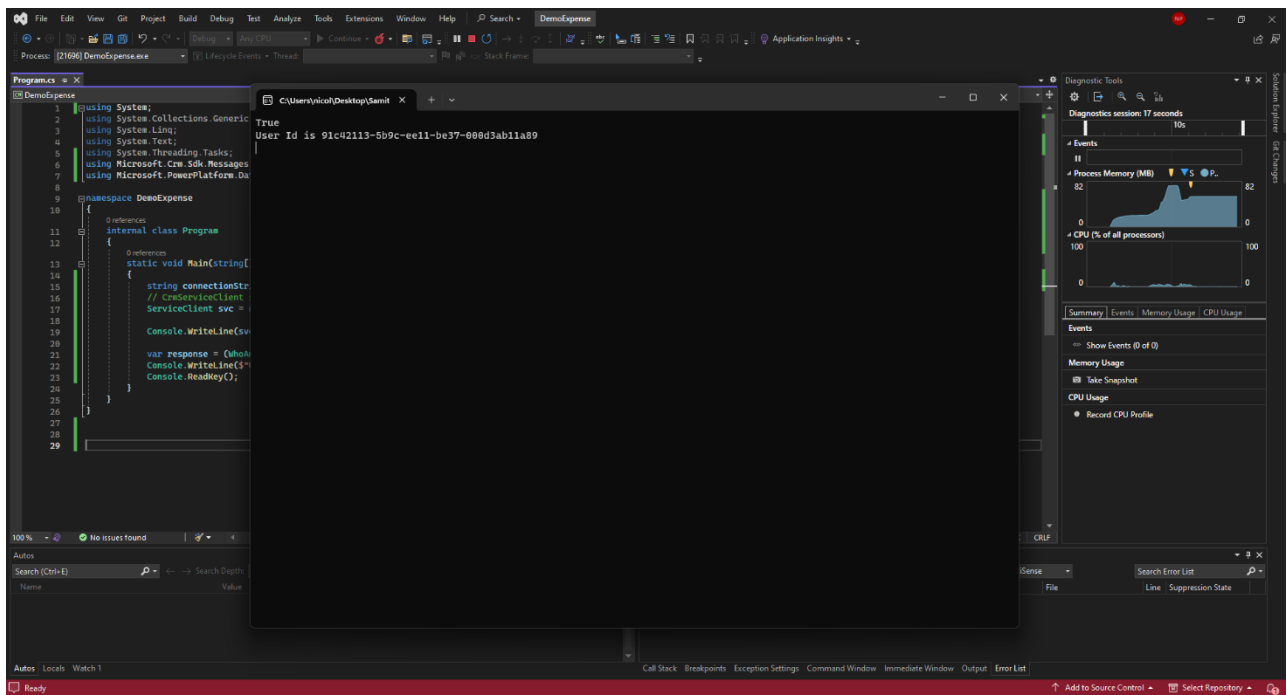


2. Create a record using console apps

We now see how to create a new contact in Dynamics 365 from our console app. Navigate to <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/org-service/quick-start-org-service-console-app> to read the related documentation.

In connecting to Dynamics 365 from the console application we can also print the Id of the user executing the operation.





This same Id is retrieved in Dynamics 365 by means of the Dynamics 365 Power Pane extension.

User Info

Current user information

User name:

Nicola Paracone

Copy it!

User id:

{078D2193-8F70-EE11-8179-000D3AA9F90A}

Copy it!

User Roles:

System Administrator

Basic User

Sales Enterprise app access

User Teams:

org8fe6a4bc

OK

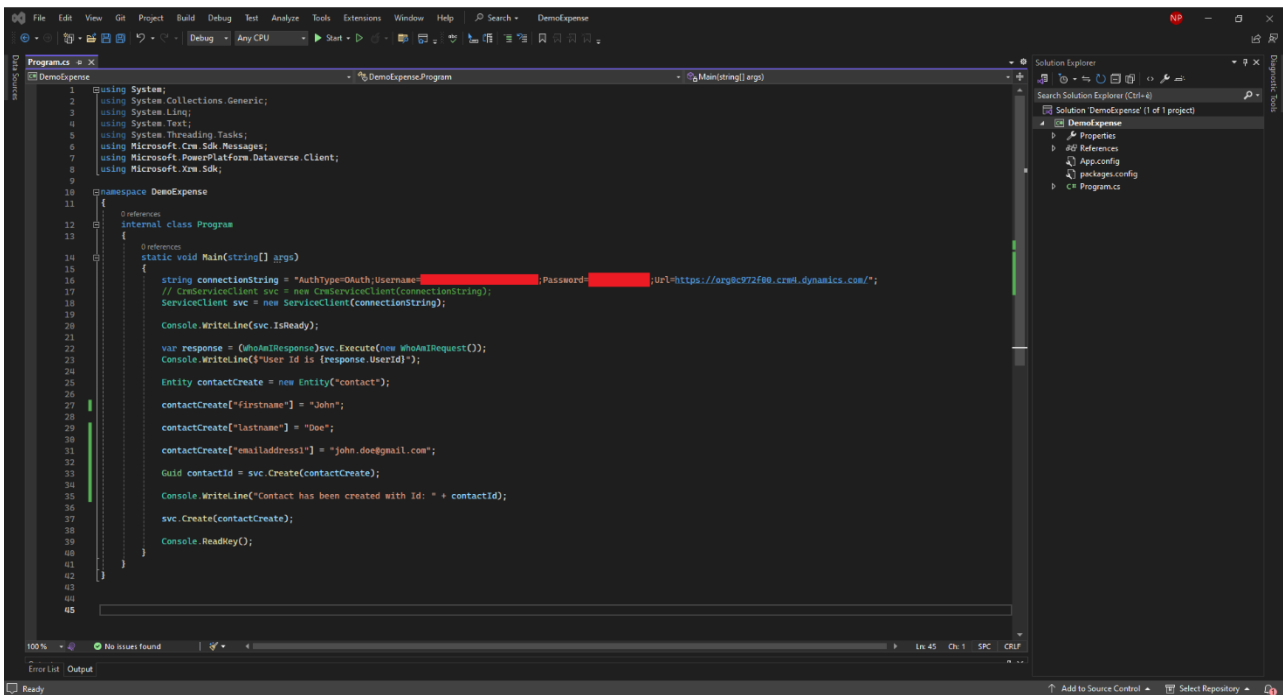
Cancel

Let us create a new contact, Jane Doe, in the Contact table.

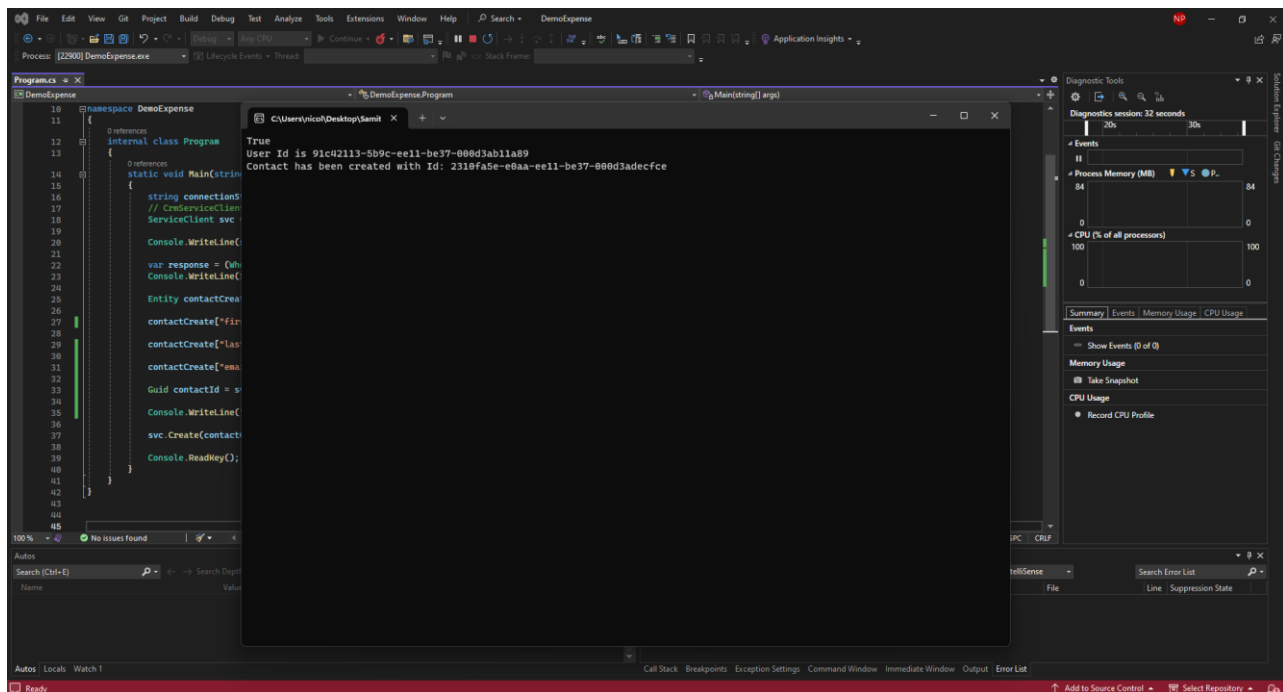
```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using Microsoft.Crm.Sdk.Messages;
7 using Microsoft.PowerPlatform.Dataaverse.Client;
8 using Microsoft.Xrm.Sdk;
9
10 namespace DemoExpense
11 {
12     internal class Program
13     {
14         static void Main(string[] args)
15         {
16             string connectionString = "AuthType=OAuth;Username=[REDACTED],Password=[REDACTED],Url=https://org0c972f00.crm.dynamics.com/";
17             // CreateServiceClient(connectionString);
18             ServiceClient svc = new ServiceClient(connectionString);
19             Console.WriteLine(svc.IsReady);
20
21             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
22             Console.WriteLine($"User Id is {response.UserId}");
23             Entity contactCreate = new Entity("contact");
24             contactCreate["firstname"] = "Jane";
25             contactCreate["lastname"] = "Doe";
26             svc.Create(contactCreate);
27             Console.ReadKey();
28         }
29     }
30 }
```

Full Name	Email	Company Name	Business Phone
Alex Baker	alex@tresearch.net	Trey Research	619-555-0127
Avery Howard	nicolaparacone@gmail.com	Trey Research	567-555-0137
Cacilia Viera	cacilia@alpineshouse.com	Alpine Ski House	281-555-0162
Carla Yates	carla@tresearch.net	Trey Research	456-555-0156
Dwayne Elijah	dwayne@alpineshouse.com	Alpine Ski House	281-555-0160
Haroun Stormonth	haroun@fabrikaminc.com	Fabrikam, Inc.	423-555-0122
Heriberto Nathan	heriberto@northwindtraders.com	Northwind Traders	614-555-0123
Jane Doe			
Kenya Brady	kenya@consolidatedmessenger.com		222-555-0172
Kevin Martin	kevin@adatum.com	A. Datum Corporation	425-555-0160
Kim Rocha	kim@tresearch.net	Trey Research	567-555-0157
Miguel Garcia	miguel@northwindtraders.com	Northwind Traders	614-555-0127
Test Postman	test_postman@gmail.com		
Test PowerAutomate	test_powerautomate@gmail.com		
Zoltan Szabo	zoltan@fabrikaminc.com	Fabrikam, Inc.	423-555-0120

Next, we create another new contact, John Doe, and also set the email address. This time, however, we additionally want to obtain the Guid of the contact.



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using Microsoft.Crm.Sdk.Messages;
7 using Microsoft.PowerPlatform.Dataaverse.Client;
8 using Microsoft.Xrm.Sdk;
9
10 namespace DemoExpense
11 {
12     [ReferenceType]
13     internal class Program
14     {
15         static void Main(string[] args)
16         {
17             string connectionString = "AuthType=OAuth;Username=[redacted];Password=[redacted];Url=https://org8c972f80.crm.dynamics.com/";
18             // CreateServiceClient svc = new CreateServiceClient(connectionString);
19             ServiceClient svc = new ServiceClient(connectionString);
20
21             Console.WriteLine(svc.IsReady);
22
23             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
24             Console.WriteLine($"User Id is {response.UserId}");
25
26             Entity contactCreate = new Entity("contact");
27
28             contactCreate["firstname"] = "John";
29             contactCreate["lastname"] = "Doe";
30             contactCreate["emailaddress1"] = "john.doe@gmail.com";
31
32             Guid contactId = svc.Create(contactCreate);
33
34             Console.WriteLine($"Contact has been created with Id: " + contactId);
35
36             svc.Create(contactCreate);
37
38             Console.ReadKey();
39         }
40     }
41 }
```



```
10 namespace DemoExpense
11 {
12     [ReferenceType]
13     internal class Program
14     {
15         static void Main(string[] args)
16         {
17             string connectionString = "AuthType=OAuth;Username=[redacted];Password=[redacted];Url=https://org8c972f80.crm.dynamics.com/";
18             // CreateServiceClient svc = new CreateServiceClient(connectionString);
19             ServiceClient svc = new ServiceClient(connectionString);
20
21             Console.WriteLine(svc.IsReady);
22
23             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
24             Console.WriteLine($"User Id is {response.UserId}");
25
26             Entity contactCreate = new Entity("contact");
27
28             contactCreate["firstname"] = "John";
29             contactCreate["lastname"] = "Doe";
30             contactCreate["emailaddress1"] = "john.doe@gmail.com";
31
32             Guid contactId = svc.Create(contactCreate);
33
34             Console.WriteLine($"Contact has been created with Id: " + contactId);
35
36             svc.Create(contactCreate);
37
38             Console.ReadKey();
39         }
40     }
41 }
```

Output:

```
True
User Id is 91c42113-5b9c-e011-be37-000d3ab11a89
Contact has been created with Id: 2310fa5e-e0aa-e011-be37-000d3adecfce
```

Dynamics 365 Expense System

Try the new look

My Active Contacts

Full Name	Email	Company Name	Business Phone
Alex Baker	alex@tresearch.net	Trey Research	619-555-0127
Avery Howard	nicolaparacone@gmail.com	Trey Research	567-555-0137
Cacilia Viera	cacilia@alpineshouse.com	Alpine Ski House	281-555-0162
Carla Yates	carla@tresearch.net	Trey Research	456-555-0156
Dwayne Elijah	dwayne@alpineshouse.com	Alpine Ski House	281-555-0160
Haroun Stormonth	haroun@fabrikaminc.com	Fabrikam, Inc.	423-555-0122
Heriberto Nathan	heriberto@northwindtraders.com	Northwind Traders	614-555-0123
Jane Doe			
John Doe	john.doe@gmail.com		
Kenya Brady	kenya@consolidatedmessenger.com		222-555-0172
Kevin Martin	kevin@adatum.com	A. Datum Corporation	425-555-0160
Kim Rocha	kim@tresearch.net	Trey Research	567-555-0157
Miguel Garcia	miguel@northwindtraders.com	Northwind Traders	614-555-0127
Test Postman	test_postman@gmail.com		
Test PowerAutomate	test_powerautomate@gmail.com		
Zoltan Szabo	zoltan@fabrikaminc.com	Fabrikam, Inc.	423-555-0120

1 - 16 of 16 (1 Selected)

3. Update a record using console apps

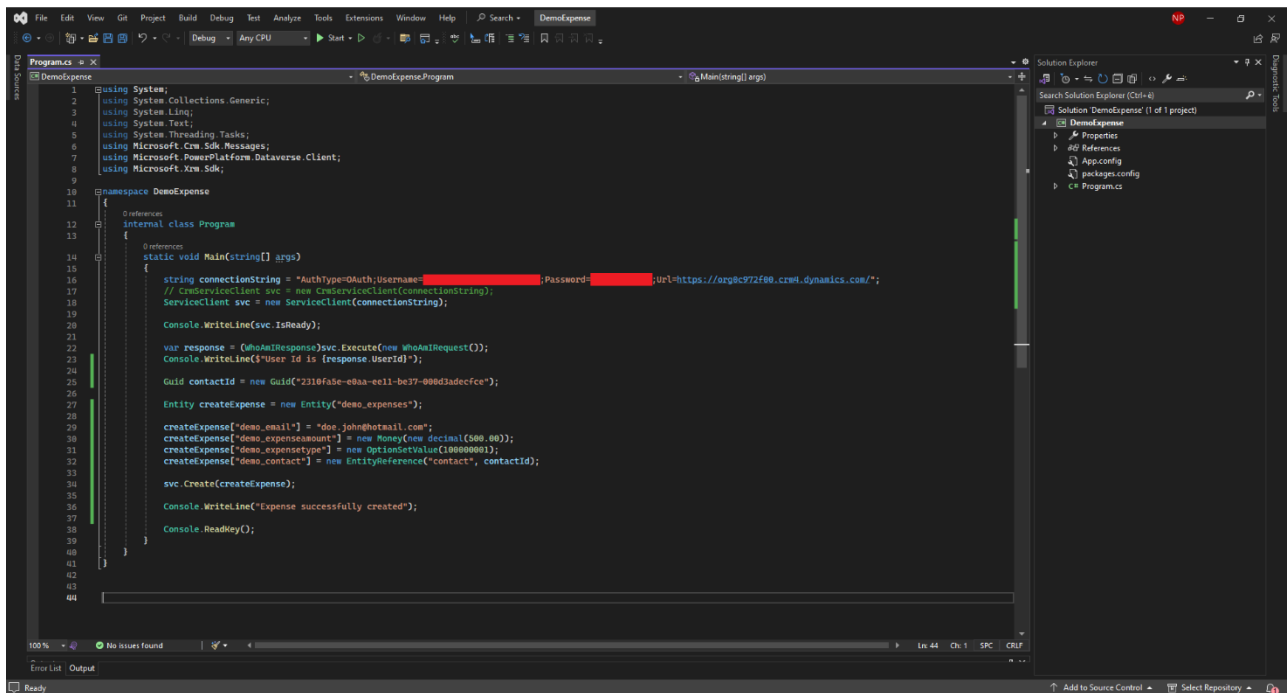
Let us update the content of the fields of a particular record having a certain Guid, for example John Doe. The following example shows how to replace the email address.

```

1  using System.Threading.Tasks;
2  using Microsoft.Crm.Sdk.Messages;
3  using Microsoft.PowerPlatform.Dataaverse.Client;
4  using Microsoft.Xrm.Sdk;
5
6  namespace DemoExpense
7  {
8      internal class Program
9      {
10         static void Main(string[] args)
11         {
12             string connectionString = "AuthType=OAuth;Username=[redacted];Password=[redacted];Url=https://orgbc972f98.crm.dynamics.com/";
13             // Create service client
14             ServiceClient svc = new ServiceClient(connectionString);
15             Console.WriteLine(svc.IsReady);
16
17             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
18             Console.WriteLine($"User Id is {response.UserId}");
19
20             Guid contactId = new Guid("2318fa5e-e8aa-e011-ba79-000d1adefcfc");
21
22             Entity contactUpdate = new Entity("contact");
23             contactUpdate.Id = contactId;
24             contactUpdate["emailaddress1"] = "doe.john@hotmail.com";
25             svc.Update(contactUpdate);
26             Console.ReadKey();
27         }
28     }
29 }

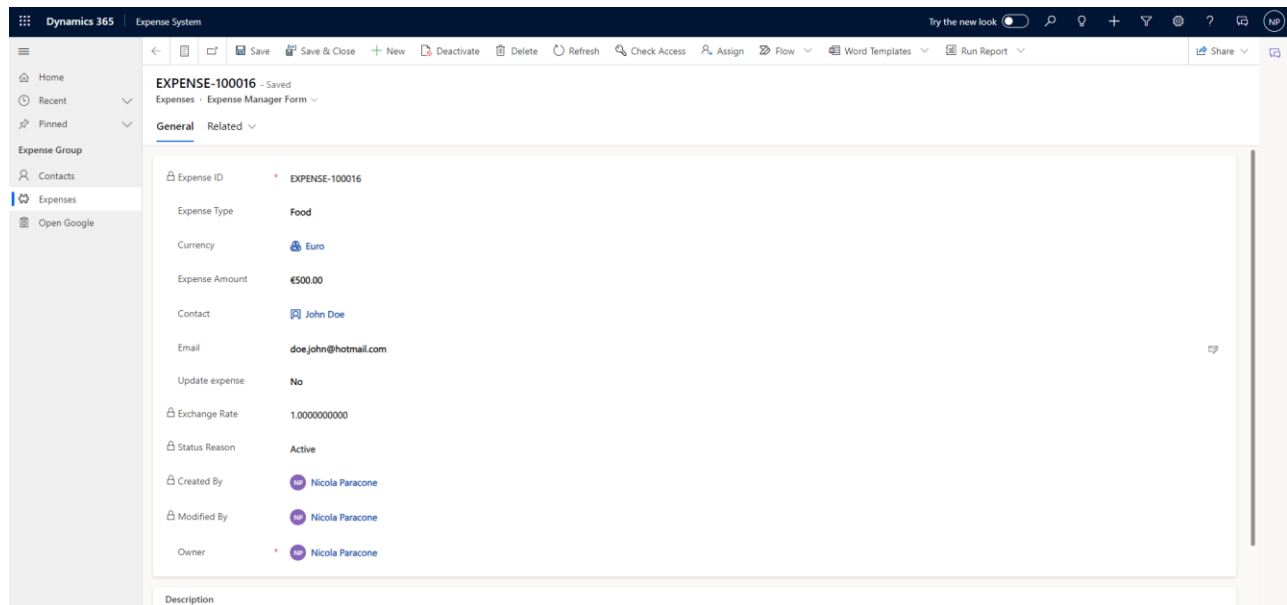
```

As a further example, we create a new expense associated with John Doe by providing the values of the required fields.



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using Microsoft.Crm.Sdk.Messages;
7 using Microsoft.PowerPlatform.Dataaverse.Client;
8 using Microsoft.Xrm.Sdk;
9
10 namespace DemoExpense
11 {
12     [Reference]
13     internal class Program
14     {
15         [Reference]
16         static void Main(string[] args)
17         {
18             string connectionString = "AuthType=OAuth;Username=[redacted];Password=[redacted];Url=https://org8c972f00.crm.dynamics.com/";
19             // CreateServiceClient svc = new CreateServiceClient(connectionString);
20             ServiceClient svc = new ServiceClient(connectionString);
21             Console.WriteLine(svc.IsReady);
22             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
23             Console.WriteLine($"User Id is {response.UserId}");
24             Guid contactId = new Guid("2310fab8-e8aa-e011-b037-000d3adecefc");
25             Entity createExpense = new Entity("demo_expenses");
26             createExpense["demo_email"] = "doe.john@hotmail.com";
27             createExpense["demo_expenseamount"] = new Money(new decimal(500.00));
28             createExpense["demo_expensetype"] = new OptionSetValue(1000000001);
29             createExpense["demo_contact"] = new EntityReference("contact", contactId);
30             svc.Create(createExpense);
31             Console.WriteLine("Expense successfully created");
32             Console.ReadKey();
33         }
34     }
35 }
```

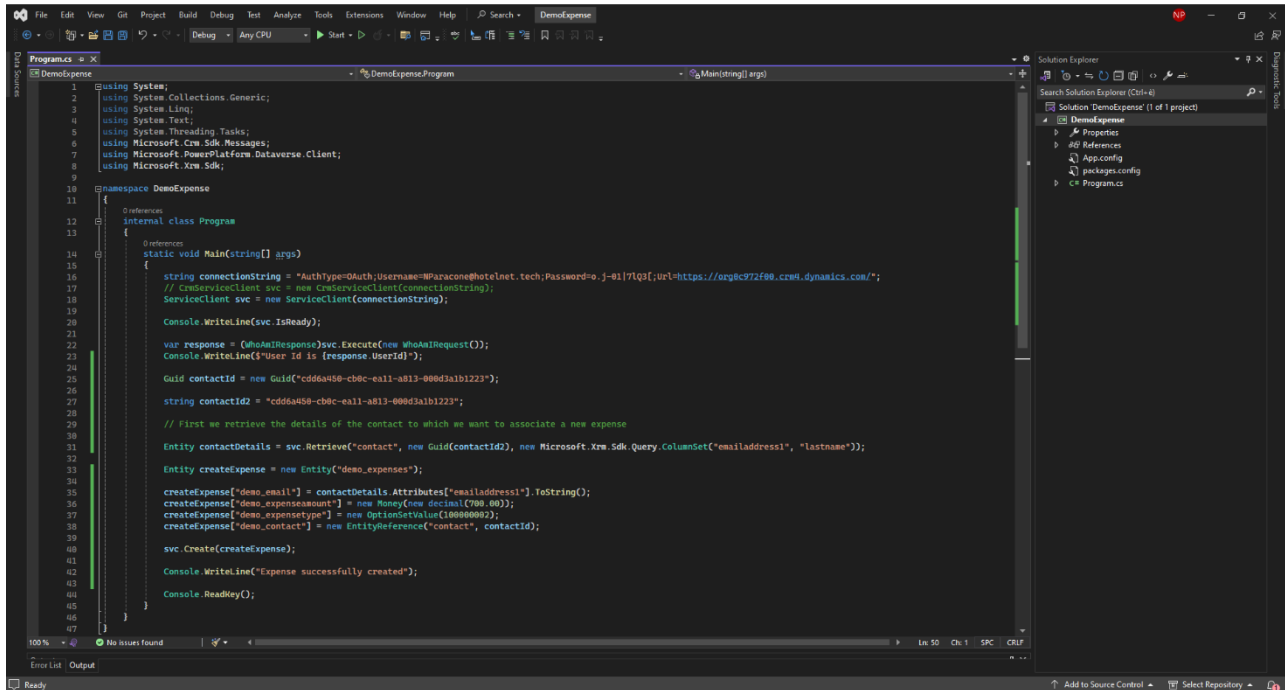
This is the newly generated expense.



EXPENSE-100016 - Saved	
Expenses - Expense Manager Form	
General Related	
Expense ID	EXPENSE-100016
Expense Type	Food
Currency	Euro
Expense Amount	€500.00
Contact	John Doe
Email	doe.john@hotmail.com
Update expense	No
Exchange Rate	1.0000000000
Status Reason	Active
Created By	Nicola Paracone
Modified By	Nicola Paracone
Owner	Nicola Paracone
Description	

4. Retrieve a record using console apps

The following example shows how to retrieve a record from the Contact table by Guid. Not all the columns are retrieved, but only the email address and the last name. The rest of the code creates another expense in which the email address is that retrieved from the associated contact record (Haroun Stormonth).

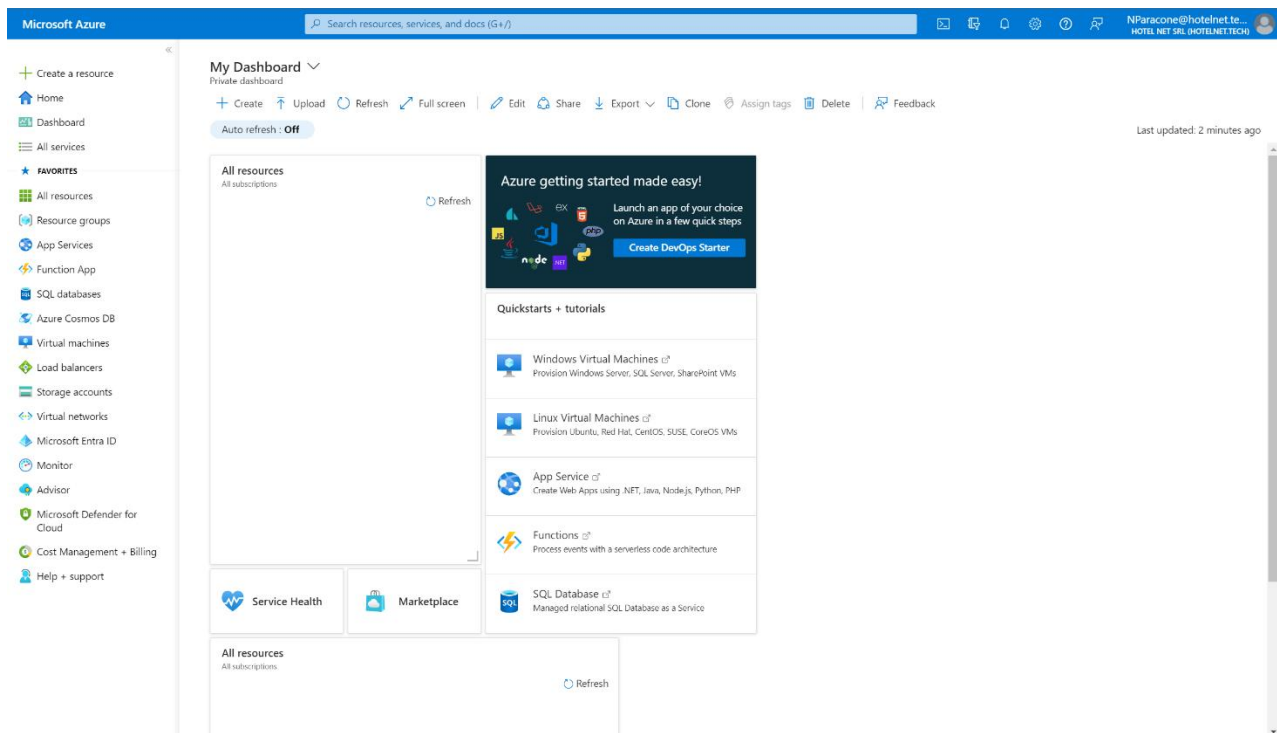


```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using Microsoft.Crm.Sdk.Messages;
7 using Microsoft.PowerPlatform.Dataaverse.Client;
8 using Microsoft.Xrm.Sdk;
9
10 namespace DemoExpense
11 {
12     // References
13     internal class Program
14     {
15         // References
16         static void Main(string[] args)
17         {
18             string connectionString = "AuthType=OAuth;Username=HParacone@hotelnet.tech;Password=o.j-B1/7lQ3f;url=https://org8c972f98.crm.dynamics.com/";
19             // CrmServiceClient svc = new CrmServiceClient(connectionString);
20             ServiceClient svc = new ServiceClient(connectionString);
21             Console.WriteLine(svc.IsReady);
22
23             var response = (WhoAmIResponse)svc.Execute(new WhoAmIRequest());
24             Console.WriteLine($"User Id is {response.UserId}");
25
26             Guid contactId = new Guid("cdd6a458-cb8c-ea11-a813-000d3a1b1223");
27             string contactId2 = "cdd6a458-cb8c-ea11-a813-000d3a1b1223";
28
29             // First we retrieve the details of the contact to which we want to associate a new expense
30
31             Entity contactDetails = svc.Retrieve("contact", new Guid(contactId2), new Microsoft.Xrm.Sdk.Query.ColumnSet("emailaddress1", "lastname"));
32
33             Entity createExpense = new Entity("demo_expenses");
34
35             createExpense["demo_email"] = contactDetails.Attributes["emailaddress1"].ToString();
36             createExpense["demo_expenseamount"] = new Money(new decimal(700.00));
37             createExpense["demo_expensetype"] = new OptionSetValue(1000000002);
38             createExpense["demo_contact"] = new EntityReference("contact", contactId);
39
40             svc.Create(createExpense);
41
42             Console.WriteLine("Expense successfully created");
43             Console.ReadKey();
44         }
45     }
46 }
47
```

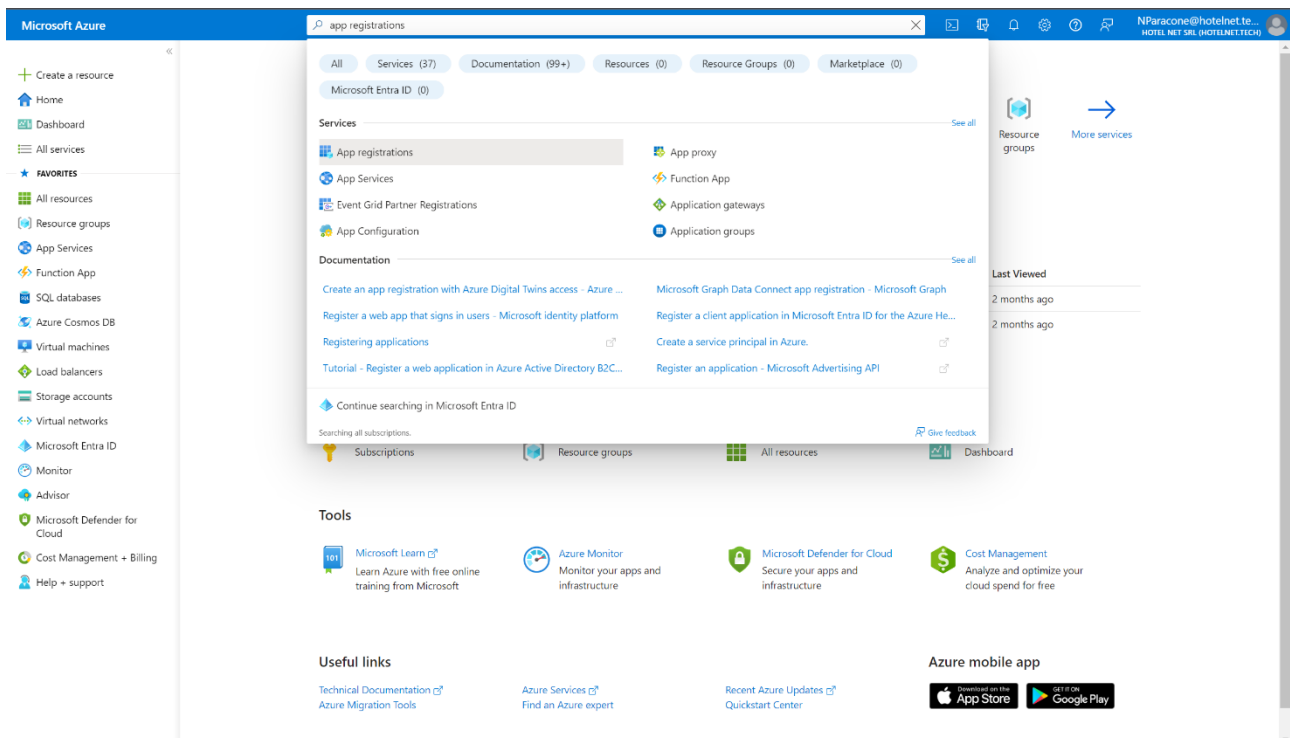
5. Connecting using an app registration

While using usernames and passwords directly in connection strings is supported, in some scenarios it is generally discouraged due to security concerns, as it exposes user credentials and increases the risk of compromise. A more secure and recommended approach is to use an app registration. That is, if we want to connect to Dynamics 365 or other Azure services, we can create an app registration in the Azure portal, obtain a client Id and client secret, and use these credentials in our application for authentication.

Navigate to portal.azure.com to access the Azure portal.



In the search bar look for App registrations.



Microsoft Azure

Search resources, services, and docs (G+/)

Dashboard >

App registrations

+ New registration | Endpoints | Troubleshooting | Refresh | Download | Preview features | Got feedback?

Starting June 30th, 2020 we will no longer add any new features to Azure Active Directory Authentication Library (ADAL) and Azure Active Directory Graph. We will continue to provide technical support and security updates but we will no longer provide feature updates. Applications will need to be upgraded to Microsoft Authentication Library (MSAL) and Microsoft Graph. [Learn more](#)

All applications | **Owned applications** | Deleted applications

Start typing a display name or application (client) ID to filter these r... | Add filters

This account isn't listed as an owner of any applications in this directory.

[View all applications in the directory](#)

Click on + New registration, choose a name, insert the required information, and click on Register.

Microsoft Azure

Search resources, services, and docs (G+/)

Dashboard > App registrations >

Register an application

* Name

The user-facing display name for this application (this can be changed later).

TrialApp ✓

Supported account types

Who can use this application or access this API?

☒ Accounts in this organizational directory only (HOTEL NET SRL only - Single tenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant)

☐ Accounts in any organizational directory (Any Microsoft Entra ID tenant - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)

Redirect URI (optional)

We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Select a platform | e.g. https://example.com/auth

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

By proceeding, you agree to the [Microsoft Platform Policies](#)

[Register](#)

Once registered, open it and go to Api permissions.

Microsoft Azure

Dashboard > App registrations > TrialApp

TrialApp | API permissions

Search resources, services, and docs (G+I)

Home

Dashboard

All services

FAVORITES

All resources

Resource groups

App Services

Function App

SQL databases

Azure Cosmos DB

Virtual machines

Load balancers

Storage accounts

Virtual networks

Microsoft Entra ID

Monitor

Advisor

Microsoft Defender for Cloud

Cost Management + Billing

Help + support

Overview

Quickstart

Integration assistant

Manage

Branding & properties

Authentication

Certificates & secrets

Token configuration

API permissions

Expose an API

App roles

Owners

Roles and administrators

Manifest

Support + Troubleshooting

Troubleshooting

New support request

The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission

Grant admin consent for HOTEL NET SRL

API / Permissions name	Type	Description	Admin consent required	Status
Microsoft Graph (1)				
User.Read	Delegated	Sign in and read user profile	No	

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

Click on + Add a permission and select Dynamics CRM.

Microsoft Azure

Dashboard > App registrations > TrialApp

TrialApp | API permissions

Search resources, services, and docs (G+I)

Home

Dashboard

All services

FAVORITES

All resources

Resource groups

App Services

Function App

SQL databases

Azure Cosmos DB

Virtual machines

Load balancers

Storage accounts

Virtual networks

Microsoft Entra ID

Monitor

Advisor

Microsoft Defender for Cloud

Cost Management + Billing

Help + support

Overview

Quickstart

Integration assistant

Manage

Branding & properties

Authentication

Certificates & secrets

Token configuration

API permissions

Expose an API

App roles

Owners

Roles and administrators

Manifest

Support + Troubleshooting

Troubleshooting

New support request

The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission

Grant admin consent for HOTEL NET SRL

API / Permissions name	Type
Microsoft Graph (1)	
User.Read	Delegated

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

Request API permissions

All APIs

Dynamics CRM

<https://admin.services.crm.dynamics.com/> Docs

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions

Start typing a permission to filter these results

The "Admin consent required" column shows the default value for an organization. However, user consent can be customized per permission, user, or app. This column may not reflect the value in your organization, or in organizations where this app will be used. [Learn more](#)

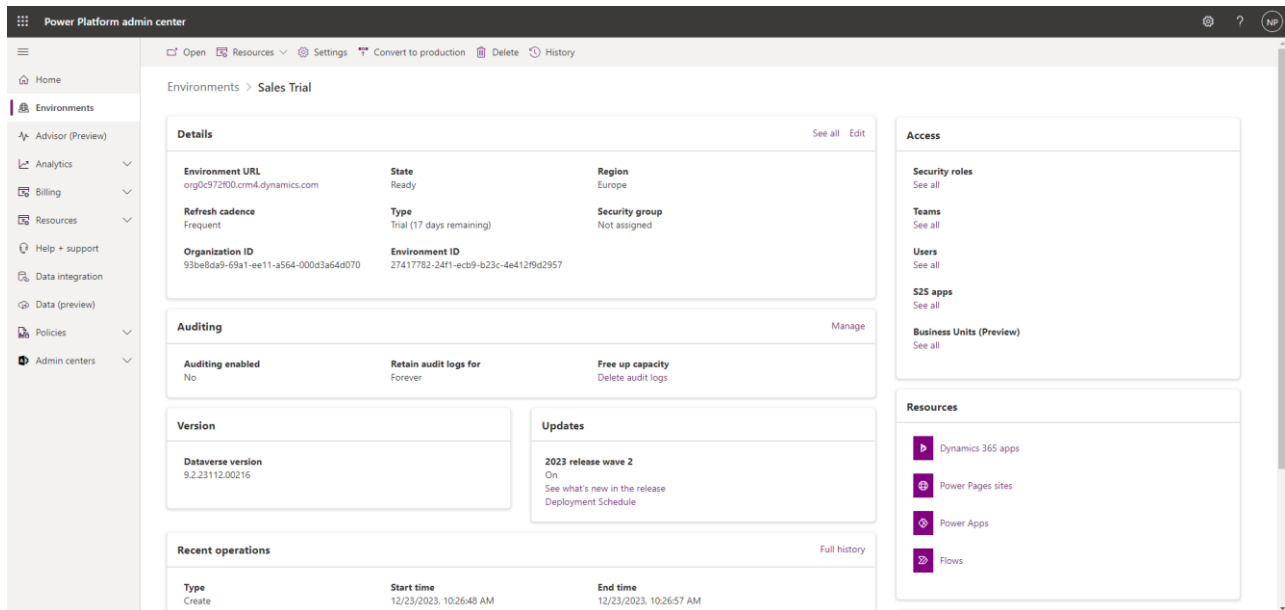
Permission	Admin consent required
user_impersonation	No

Access Common Data Service as organization users

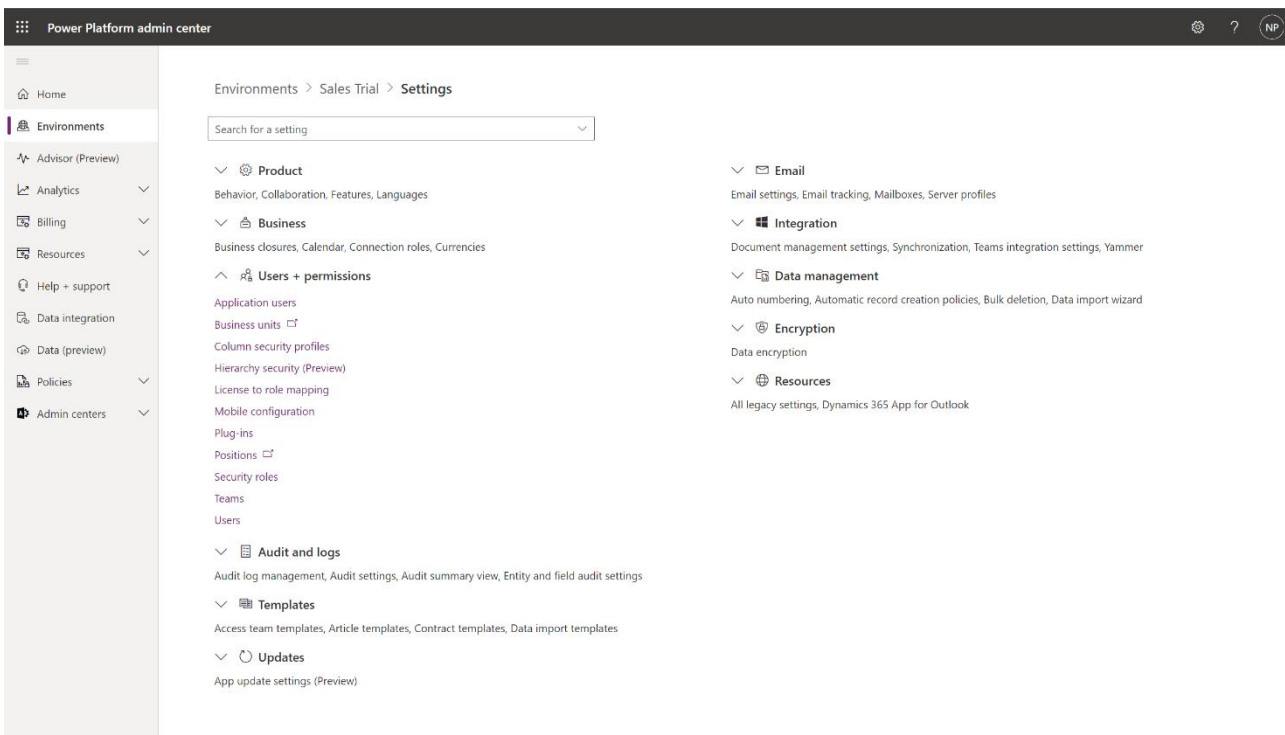
Add permissions Discard

Hence, check the user_impersonation box and click Add permissions. By adding Dataverse CRM to the API permissions, our application gains the ability to perform operations like reading, writing, updating, or deleting data in Dataverse. These permissions are essential for applications that need to integrate with or extend the functionality of Dynamics 365 or Power Platform.

At this point, we have to go to admin.powerplatform.microsoft.com and enter the desired environment to which we want to give the permissions.



Click on Settings and then on Users + permissions.



Assuming that a new employee has joined our organization, by default they will not be added to all the environments. So, we can add that user manually by clicking on Users.

Application users, instead, are accounts associated with applications, used for service-to-service authentication and automated interactions with Power Platform services. They represent the identity of an application and enable secure, permission-controlled access for background processes, automation, and integrations.

Power Platform admin center

Environments > Sales Trial > Settings > Application users

Manage the non-interactive users in this environment so your apps can access Dataverse data. [Learn more](#)

Name	App ID	State	App type	Security roles	Business unit
Power Apps Checker Application	c9299480-c13a-49db-a7ae-cdfe54fe0313	Active	Custom	3	org8f6a4bc
Microsoft Forms Pro	19dd5b37-d116-48cb-90d2-4aa56696c8a1	Active	Custom	2	org8f6a4bc

Click on + New app user and select from all the registrations the one we have just created in Azure, namely TrialApp.

Power Platform admin center

Environments > Sales Trial > Settings > Application users

Manage the non-interactive users in this environment so your apps can access Dataverse data. [Learn more](#)

Name	App ID	State	App type
Power Apps Checker Application	c9299480-c13a-49db-a7ae-cdfe54fe0313	Active	Custom
Microsoft Forms Pro	19dd5b37-d116-48cb-90d2-4aa56696c8a1	Active	Custom

Add an app from Microsoft Entra ID

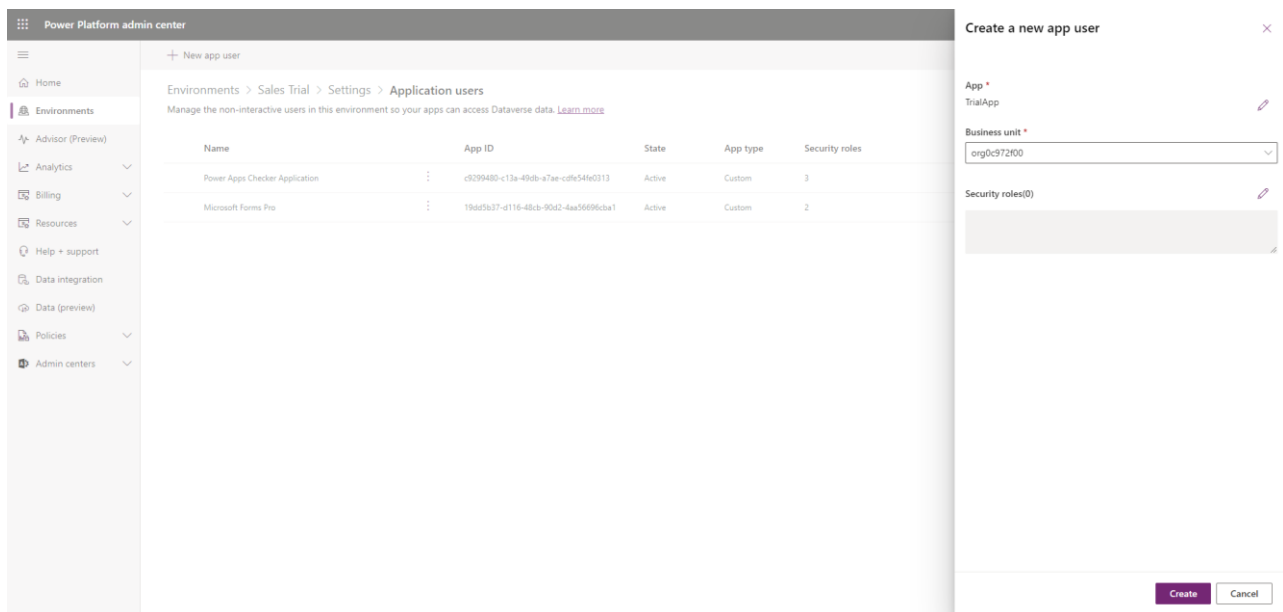
Search: TrialApp

There could be multiple reasons why your app may not show up in the list. [Learn more](#)

Name ↑	App ID
TrialApp	338f1175-186a-4...

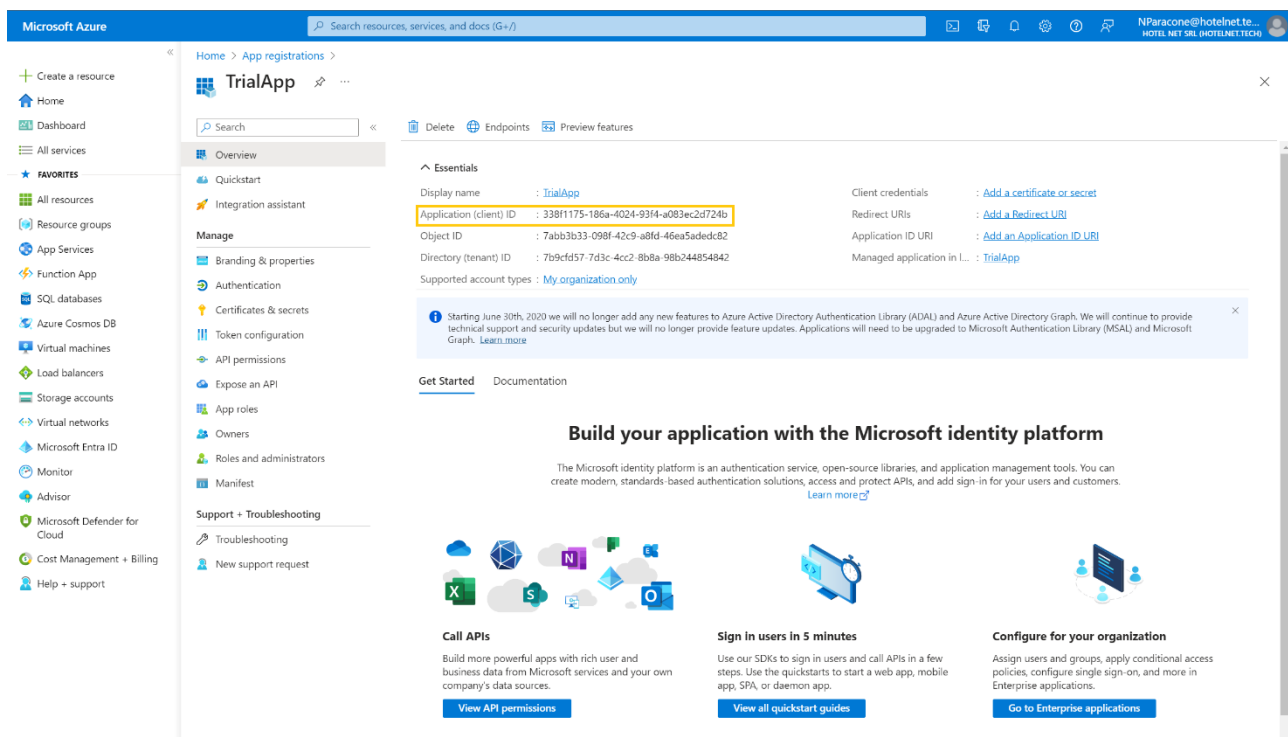
Add Cancel

Hence select the business unit and the security permissions you want to give to the app registration.



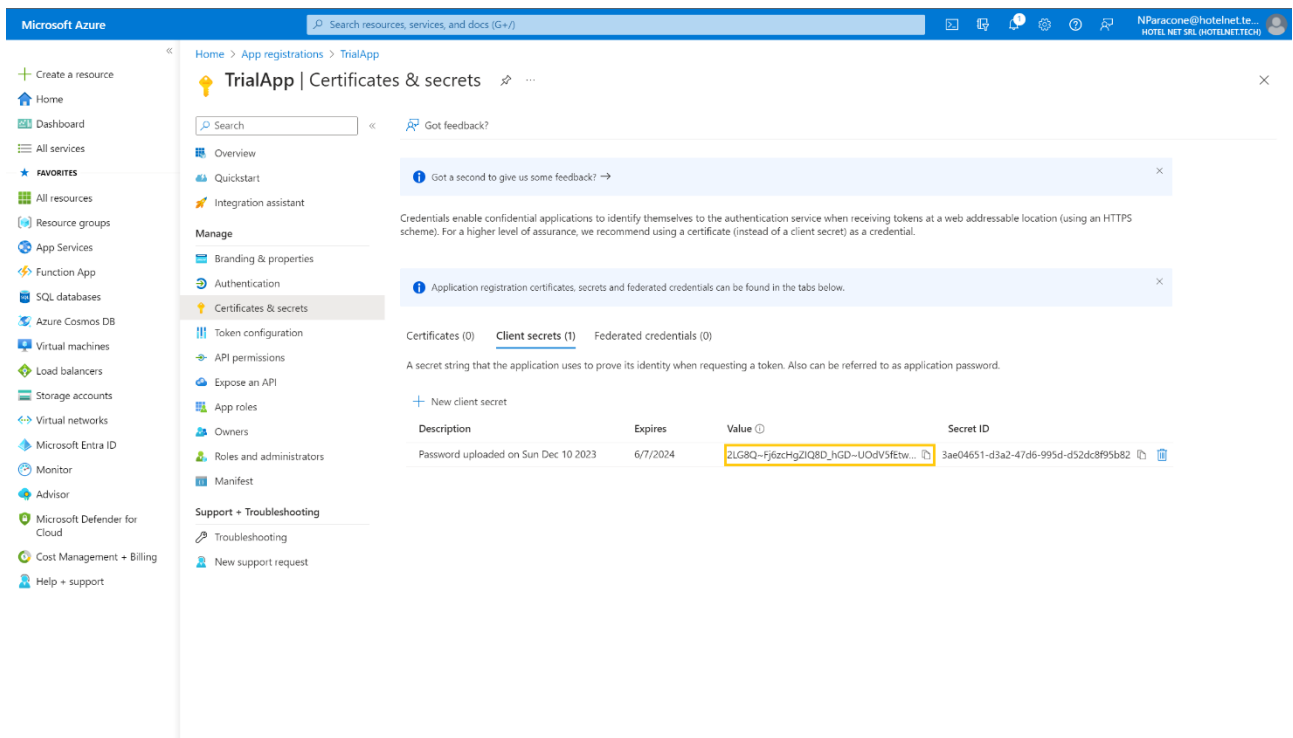
Finally, click on Create.

At this point, to use the app registration for connecting to Dynamics 365 in a console application, we need two things: the Application (client) ID and a Client secret.



Go to Certificates & secrets and click on + New client secret. The purpose of a client secret is to authenticate an application securely with a server, such as Azure Active Directory (Azure AD), during the process of obtaining an access token.

It serves as a confidential piece of information that proves the identity of the application and is used in the authentication flow.



The following is an example of console application connecting to our Sales trial environment using clientId and clientSecret. Notice that possible connection errors are handled properly.

